

Tema 3. Parte 1

Trabajo con ficheros XML

Acceso a datos.

DOM Y SAX

1. Acceso a datos con DOM y SAX

DOM
(Document Object Model)

SAX
(Simple Api for XML)

Leer (SAX y DOM) y escribir (DOM) ficheros XML

Detectar elementos de un fichero XML

Verificar si los ficheros son válidos

1.Ventajas y desventajas: DOM

DOM	
Ventaja	Desventaja
Al leer el fichero crea un árbol formado por nodos en memoria.	Almacena todo el fichero en memoria
	Más lento que SAX

1. Ventajas y desventajas: SAX

SAX

Ventaja

Lee el fichero línea a línea, sin cargar todo el documento en memoria.

Desventaja

Menos potente que DOM al no conocer todo el documento.

1. Recomendación de uso

DOM

Para tareas complejas que requieran conocer todo el fichero y tengamos un objetivo claro.

SAX

Para recorrer ficheros secuencialmente y realizar operaciones simples

1. Comparación

SAX

Basado en eventos (dispara eventos para sabes cuando inicia y cuando termina una etiqueta XML)

Analiza nodo a nodo:

`<persona
edad="30">Juan</persona>`

DOM

Carga todo el fichero en memoria

Permite buscar tags tanto hacia delante y hacia detrás en todo el documento

1. Comparación

SAX

Analiza sobre la marcha sin cargar el fichero en memoria

Rápido en tiempo de ejecución

DOM

Almacena el fichero en memoria en forma de árbol.

Lento en tiempo de ejecución

1. Comparación

SAX

Solo permite lectura de ficheros XML.

DOM

Se pueden insertar y/o eliminar nodos

Conversión de ficheros XML

2.Conversión de ficheros XML

Existen muchas librerías que sirven como parsers de ficheros XML

Java:

`javax.xml.parsers`

Ejemplo paseador de tipo DOM

2.1 Ejemplo de paseador DOM (Crear instancia)

```
public static void main(String[] args) {  
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();  
    factory.setValidating(true);  
    factory.setIgnoringElementContentWhitespace(true);  
  
    try {  
        DocumentBuilder builder = factory.newDocumentBuilder();  
        File file = new File("tema3_1.xml");  
        Document doc = builder.parse(file);  
    } catch (ParserConfigurationException | SAXException | IOException e) {  
        e.printStackTrace();  
    }  
}
```

2.1 Ejemplo de paseador DOM (Crear instancia)

Crear una instancia:

```
DocumentBuilderFactory factory =  
DocumentBuilderFactory.newInstance();
```

Validar documento:

```
factory.setValidating(true);  
Si no es válido lanza una excepción.
```

Ignorar elementos innecesarios

```
factory.setIgnoringElementContentWhitespace(true);
```


2.1 Ejemplo de paseador DOM (Crear instancia)

```
<libro>  
  <titulo>XML Simplificado</titulo>  
  <autor>Juan Pérez</autor>  
</libro>
```


2.1 Ejemplo de paseador DOM (Crear instancia)

```
factory.setIgnoringElementContentWhitespace(true);
```

```
libro
├── titulo
│   └── "XML Simplificado"
└── autor
    └── "Juan Pérez"
```

```
factory.setIgnoringElementContentWhitespace(false);
```

```
libro
├── #text (espacio o salto de línea después de <libro>)
├── titulo
│   ├── #text (espacios antes del contenido "XML Simplificado")
│   │   └── "XML Simplificado"
│   └── #text (espacios después del contenido "XML Simplificado")
├── #text (espacio o salto de línea después de </titulo>)
├── autor
│   ├── #text (espacios antes de "Juan Pérez")
│   │   └── "Juan Pérez"
│   └── #text (espacios después de "Juan Pérez")
└── #text (espacio o salto de línea después de </autor>)
```


2.1 Ejemplo de paseador DOM (Crear instancia)

```
public static void main(String[] args) {  
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();  
    factory.setValidating(true);  
    factory.setIgnoringElementContentWhitespace(true);  
  
    try {  
        DocumentBuilder builder = factory.newDocumentBuilder();  
        File file = new File("tema3_1.xml");  
        Document doc = builder.parse(file);  
    } catch (ParserConfigurationException | SAXException | IOException e) {  
        e.printStackTrace();  
    }  
}
```

2.1 Ejemplo de paseador DOM (Crear instancia)

DocumentBuilder

Permite obtener un dato de tipo Document a partir de un documento XML para ya realizar operaciones

```
DocumentBuilder builder =  
factory.newDocumentBuilder()
```

```
File file = new File("tema3_1.xml");
```

Leer el fichero
XML.

```
Document doc = builder.parse(file);
```

Ejemplo paseador de tipo SAS

2.1 Ejemplo de paseador DOM (Crear instancia)

javax.xml.parsers

SAXParserFactory &
SAXParser

```
public static void main(String[] args) {  
    SAXParserFactory factory = SAXParserFactory.newInstance();  
    factory.setValidating(true);  
    try {  
        SAXParser saxParser = factory.newSAXParser();  
        File file = new File("tema3_1.xml");  
        saxParser.parse(file, new DefaultHandler());  
    } catch (ParserConfigurationException | SAXException | IOException e) {  
        e.printStackTrace();  
    }  
}
```

2.2 Ejemplo de paseador SAX (Crear instancia)

Crear una
instancia:

```
SAXParserFactory factory =  
SAXParserFactory.newInstance();
```

Validar
documento:

```
factory.setValidating(true);
```


2.2 Ejemplo de paseador SAX (Crear instancia)

javax.xml.parsers

SAXParserFactory &
SAXParser

```
public static void main(String[] args) {  
    SAXParserFactory factory = SAXParserFactory.newInstance();  
    factory.setValidating(true);  
    try {  
        SAXParser saxParser = factory.newSAXParser();  
        File file = new File("tema3_1.xml");  
        saxParser.parse(file, new DefaultHandler());  
    } catch (ParserConfigurationException | SAXException | IOException e) {  
        e.printStackTrace();  
    }  
}
```

2.2 Ejemplo de paseador SAX (Crear instancia)

Crear una
instancia
SAXParser:

```
SAXParser saxParser = factory.newSAXParser();
```

Utilizado para analizar el archivo XML.

Leer el fichero
XML:

```
File file = new File("tema3_1.xml");  
saxParser.parse(file, new DefaultHandler());
```

No necesita objeto de tipo Document porque no se almacena en memoria.

Procesamiento de ficheros XML

Contenido teórico en la práctica 3.1

Agradecimientos

**Material adaptado a partir del trabajo de:
Antonio Cuadros Lapresa**

“El conocimiento es poder, la información es liberadora” - Kofi Annan